

joyerIA — Engineering Workflow & Development Standards

1. INFORMACIÓN GENERAL

Proyecto

joyerIA

Documento

Engineering Workflow & Development Standards

Objetivo

Definir el flujo oficial de trabajo técnico y estándares de ingeniería para el desarrollo de joyerIA.

Este documento establece:

- Git workflow.
- Branching strategy.
- Pull requests.
- Commits.
- Versionado.
- API versioning.
- Code review.
- CI/CD.
- Releases.
- Calidad técnica.
- Convenciones.
- Entornos.

2. PRINCIPIOS DE DESARROLLO

Filosofía principal

El proyecto debe priorizar:

- Escalabilidad.

- Mantenibilidad.
 - Simplicidad.
 - Modularidad.
 - Calidad.
 - Desarrollo iterativo.
 - Bajo acoplamiento.
-

Reglas fundamentales

Nunca

- Hacer push directo a production.
 - Romper compatibilidad sin versionado.
 - Mezclar features grandes en un solo PR.
 - Subir secretos al repositorio.
 - Crear archivos gigantes.
 - Acoplar módulos innecesariamente.
-

Siempre

- Crear ramas por feature.
 - Hacer PRs pequeños.
 - Revisar código.
 - Mantener tipado fuerte.
 - Documentar cambios importantes.
 - Mantener arquitectura limpia.
-

3. GIT WORKFLOW

Estrategia oficial

El proyecto utilizará:

Trunk-Based Development simplificado

Con:

- main.
- develop.
- feature branches.

- hotfix branches.
-

Branches principales

main

Representa producción estable.

Reglas:

- Nunca desarrollo directo.
 - Solo código validado.
 - Siempre deployable.
-

develop

Branch principal de integración.

Reglas:

- Integración de features.
 - Ambiente staging.
 - QA testing.
-

Branch naming conventions

Features

```
feature/feed-video-player  
feature/ai-image-generation  
feature/chat-realtime
```

Fixes

```
fix/feed-scroll-lag  
fix/chat-message-order
```

Hotfixes

hotfix/auth-token-expiration

Refactors

refactor/feed-module-cleanup

Chores

chore/update-dependencies

Workflow oficial

```
Create Branch
↓
Develop Feature
↓
Local Testing
↓
Pull Request
↓
Code Review
↓
CI Validation
↓
Merge into develop
↓
Testing
↓
Release into main
```

4. PULL REQUEST STANDARDS

Reglas obligatorias

Cada PR debe:

- Tener objetivo claro.
 - Ser pequeño/modular.
 - Compilar correctamente.
 - Pasar linting.
 - Pasar tests.
 - Tener descripción.
-

Tamaño recomendado

Ideal:

100 - 400 líneas

Evitar:

2000+ líneas

Template PR

Debe incluir

Objetivo
Cambios realizados
Riesgos
Testing realizado
Screenshots/videos

Reglas merge

Requiere

- CI passing.
 - Review aprobado.
 - Sin conflictos.
 - Sin errores críticos.
-

5. COMMIT STANDARDS

Convención oficial

Conventional Commits

Tipos permitidos

feat

Nueva funcionalidad.

```
feat(feed): add infinite scroll pagination
```

fix

Corrección de bug.

```
fix(chat): resolve realtime duplicate messages
```

refactor

Refactor interno.

```
refactor(auth): simplify session validation
```

chore

Tareas técnicas.

```
chore: update dependencies
```

docs

Documentación.

```
docs: update architecture documentation
```

style

Cambios visuales sin lógica.

```
style(feed): improve spacing and typography
```

test

Testing.

```
test(chat): add message repository tests
```

Reglas commits

Nunca

```
fix stuff  
last changes  
final final
```

Siempre

Mensajes claros y específicos.

6. VERSIONADO

Estrategia oficial

Semantic Versioning (SemVer)

Formato:

```
MAJOR.MINOR.PATCH
```

Ejemplo:

```
1.2.4
```

Significado

MAJOR

Cambios incompatibles.

MINOR

Nuevas funcionalidades compatibles.

PATCH

Correcciones pequeñas.

Ejemplos

```
1.0.0 → lanzamiento MVP
1.1.0 → nueva feature IA
1.1.1 → fix chat
2.0.0 → breaking changes API
```

7. API VERSIONING

Estrategia oficial

Toda API pública debe versionarse.

Formato oficial

```
/api/v1/
/api/v2/
```

Ejemplo

```
/api/v1/feed
/api/v1/chat
/api/v1/ai/generate
```

Reglas importantes

Nunca

- Romper contratos existentes silenciosamente.
 - Cambiar respuestas sin versionado.
-

Siempre

- Mantener backward compatibility.
 - Documentar breaking changes.
-

Estrategia evolución

v1

MVP.

v2

Cambios estructurales importantes.

8. CODE REVIEW STANDARDS

Objetivo

Garantizar:

- Calidad.
 - Seguridad.
 - Performance.
 - Mantenibilidad.
-

Checklist review

Arquitectura

- Respetar Clean Architecture.
 - Respetar modularidad.
 - No romper separación de capas.
-

Código

- Legible.
 - Tipado.
 - Sin duplicación.
 - Nombres claros.
-

Seguridad

- Validaciones.
 - Sin secretos hardcoded.
 - Políticas correctas.
-

Performance

- Sin renders innecesarios.
 - Sin queries pesadas.
 - Sin loops problemáticos.
-

UX

- Loading states.
 - Error states.
 - Responsive.
-

9. CI/CD

Objetivo

Automatizar:

- linting.
 - testing.
 - builds.
 - deploys.
-

Pipeline mínimo

```
Push
↓
Lint
↓
Tests
↓
Build
↓
Deploy Staging
↓
QA
↓
Deploy Production
```

Herramienta oficial

GitHub Actions

Validaciones automáticas

Frontend

- flutter analyze.
- formatting.

- tests.
-

Backend

- linting.
 - type checks.
 - tests.
-

10. ENVIRONMENTS

Ambientes oficiales

Development

Uso local.

Staging

QA/testing.

Production

Usuarios reales.

Variables de entorno

Nunca

Subir:

- API keys.
 - passwords.
 - tokens.
-

Siempre

Usar:

```
.env  
secret manager  
CI secrets
```

11. TESTING STRATEGY

Pirámide testing

```
Unit Tests  
↓  
Integration Tests  
↓  
E2E Critical Flows
```

Prioridades MVP

Crítico

- Auth.
 - Feed.
 - Chat.
 - Payments futuros.
 - IA.
-

Herramientas sugeridas

Flutter

```
flutter_test
mocktail
integration_test
```

12. DOCUMENTATION STANDARDS

Documentación obligatoria

Arquitectura

Actualizar cuando:

- cambien módulos.
- cambie estructura.
- cambie flujo.

APIs

Documentar:

- endpoints.
- payloads.
- responses.
- errores.

README

Debe incluir:

- setup.
 - arquitectura.
 - scripts.
 - comandos.
-

13. DEPENDENCY MANAGEMENT

Reglas

Nunca

- instalar paquetes innecesarios.
 - usar librerías abandonadas.
 - duplicar funcionalidades.
-

Siempre

- validar mantenimiento.
 - validar comunidad.
 - revisar seguridad.
-

Estrategia updates

Dependencias críticas

Actualizar mensualmente.

Dependencias menores

Actualizar gradualmente.

14. SECURITY WORKFLOW

Reglas obligatorias

Auth

- Tokens seguros.
 - Expiración.
 - Refresh strategy.
-

Storage

- Validar uploads.
 - Policies.
 - Access control.
-

Database

- RLS obligatorio.
 - Políticas explícitas.
-

APIs

- Rate limiting.
 - Validación payloads.
 - Sanitización.
-

15. RELEASE STRATEGY

Estrategia oficial

Releases pequeñas y frecuentes.

Evitar:

mega releases

Flujo release

```
develop
↓
staging
↓
QA
↓
main
```

↓
production

Rollback strategy

Debe existir capacidad de:

- rollback rápido.
 - feature disabling.
 - hotfix deployment.
-

16. OBSERVABILIDAD Y MONITOREO

Logs importantes

Registrar:

- auth failures.
 - server errors.
 - AI errors.
 - payment failures.
 - crashes.
-

Métricas críticas

Performance

- app startup.
 - API latency.
 - feed FPS.
-

Negocio

- DAU.
- retención.
- engagement.
- IA usage.

Herramientas sugeridas

Monitoring

- PostHog.
- Crashlytics.
- Supabase logs.

17. ARQUITECTURA DE ESCALABILIDAD FUTURA

MVP

Monolito modular.

Crecimiento

Extraer:

- AI workers.
- recommendation engine.
- video processing.

Regla importante

No introducir:

- microservicios.
- event buses complejos.
- Kubernetes.

Hasta tener:

- PMF.
 - crecimiento real.
 - necesidad técnica comprobada.
-

18. PRINCIPIOS FINALES DE INGENIERÍA

Prioridades reales

Más importante que:

- microservicios.
 - arquitectura hype.
 - complejidad enterprise.
-

Es:

- código limpio.
 - modularidad.
 - UX excelente.
 - velocidad de iteración.
 - arquitectura mantenible.
 - observabilidad.
 - calidad técnica.
-

19. CONCLUSIÓN

El flujo de trabajo y estándares de ingeniería de joyería están diseñados para permitir:

- Desarrollo rápido.
- Escalabilidad progresiva.
- Excelente mantenibilidad.
- Calidad técnica profesional.
- Bajo costo operativo.
- Evolución segura del producto.

La prioridad principal será construir un producto estable, modular y preparado para crecer sin introducir complejidad prematura.